

Contoh	22	10	15	3	8	2
Iterasi 1						
Langkah 1	1	2	3	4	5	6
	22	10	15	3	8	2
	LB					UB
Langkah 2	2	10	15	3	8	22
Iterasi 2						
Langkah 1	2	10	15	3	8	22
	LB/UB					
Langkah 2	2	10	15	3	8	22
		LB			UB	
Iterasi 3						
Langkah 1	2	10	15	3	8	22
		LB			UB	
Langkah 2	2	8	15	3	10	22
Iterasi 4						
Langkah 1	2	8	15	3	10	22
		LB		UB		
Langkah 2	2	3	15	8	10	22
Iterasi 5						
Langkah 1	2	3	15	8	10	22
			LB		UB	
Langkah 2	2	3	10	8	15	22
Iterasi 6						
Langkah 1	2	3	10	8	15	22
			LB	UB		
Langkah 2	2	3	8	10	15	22

Analisis Tabel Simulasi Quick Sort

Tabel tersebut mendokumentasikan proses pengurutan array [22, 10, 15, 3, 8, 2] menggunakan metode partisi. Berikut poin-poin penjelasannya:

1. **Fungsi LB dan UB:** LB (Lower Bound) menandakan batas awal sub-array yang sedang diproses, sedangkan UB (Upper Bound) adalah batas akhirnya.
2. **Proses Iterasi:**
 - a. **Iterasi 1:** Elemen pertama (**22**) ditetapkan sebagai pivot. Algoritma melakukan pengecekan dari kedua arah untuk menempatkan elemen sesuai

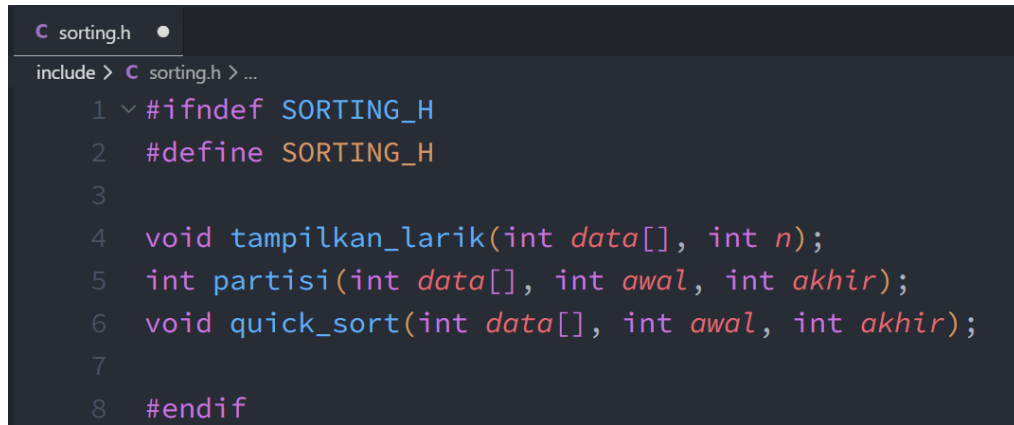
urutan. Hasil akhirnya, angka **22** dipindah ke posisi paling kanan (posisi ke-6) karena merupakan nilai terbesar.

- b. **Iterasi 2 hingga 6:** Algoritma masuk ke proses rekursif pada sub-array yang tersisa (posisi 1 sampai 5). Fokusnya adalah memecah bagian yang belum terurut hingga semua elemen berada di posisi yang benar.
3. **Hasil Akhir:** Setelah 6 iterasi, data berhasil disusun ulang dari [22, 10, 15, 3, 8, 2] menjadi urutan *ascending* yaitu [2, 3, 8, 10, 15, 22].

Tabel ini menunjukkan cara kerja partisi di mana setiap langkah bertujuan untuk menempatkan satu atau lebih elemen ke posisi yang tepat sebelum memproses bagian array yang lebih kecil.

PENJELASAN KODE PROGRAMNYA :

1. Penjelasan File Header (sorting.h)

A screenshot of a code editor window titled 'C sorting.h'. The editor shows the following C code:

```
include > C sorting.h > ...
1  ~ #ifndef SORTING_H
2    #define SORTING_H
3
4  void tampilkan_larik(int data[], int n);
5  int partisi(int data[], int awal, int akhir);
6  void quick_sort(int data[], int awal, int akhir);
7
8  #endif
```

File `sorting.h` berfungsi sebagai *header file* yang mendefinisikan struktur antarmuka (*interface*) bagi fungsi-fungsi yang digunakan dalam program ini. Berikut adalah rincian komponen di dalamnya:

1. **Header Guard (#ifndef dan #define):** Bagian ini berfungsi untuk mencegah *double inclusion* atau pemanggilan berulang pada file yang sama selama proses kompilasi, sehingga menjaga stabilitas struktur proyek.
2. **void tampilkan_larik(int data[], int n);:** Fungsi ini bertujuan untuk mencetak elemen-elemen array ke layar. Fungsi ini sangat krusial untuk memantau perubahan data sebelum dan sesudah proses pengurutan.
3. **int partisi(int data[], int awal, int akhir);:** Fungsi ini merupakan inti logika dari algoritma *Quick Sort*. Tugasnya adalah mengatur posisi elemen berdasarkan nilai *pivot* dan mengembalikan posisi indeks (*pivot point*) yang telah terurut.
4. **void quick_sort(int data[], int awal, int akhir);:** Fungsi ini mengimplementasikan logika *Quick Sort* secara rekursif. Fungsi ini membagi array menjadi bagian-bagian yang lebih kecil secara bertahap hingga seluruh data mencapai kondisi terurut.

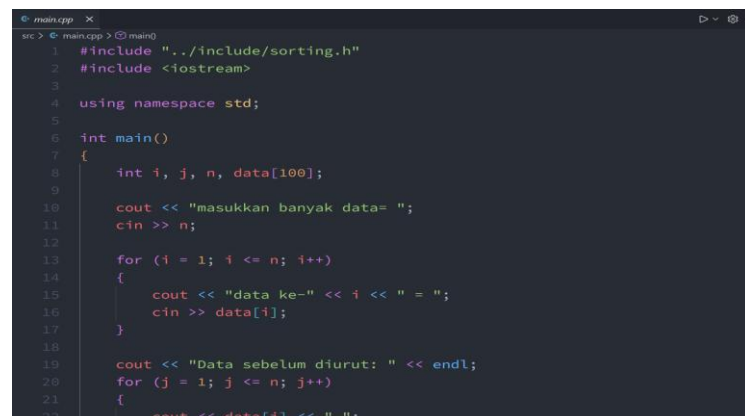
2. Penjelasan *Source Code* (sorting.cpp)

```
src > sorting.cpp x
src > sorting.cpp > quick_sort(int [], int, int)
1  #include "../include/sorting.h"
2  #include <iostream>
3
4  using namespace std;
5
6  void tampilkan_larik(int data[], int n)
7  {
8      int i;
9      for (i = 1; i <= n; i++)
10         cout << data[i] << " ";
11     cout << "\n";
12 }
13
14 int partisi(int data[], int awal, int akhir)
15 {
16     int i, j, simpan;
17     i = awal;
18     j = akhir;
19     while (1)
20     {
21         while (data[i] < data[awal])
22             i = i + 1;
23
24         while (data[j] > data[awal])
25             j = j - 1;
26
27         if (i < j)
28         {
29             // tukarkan data
30             simpan = data[i];
31             data[i] = data[j];
32             data[j] = simpan;
33         }
34         else
35             return j;
36     }
37 }
38
39 void quick_sort(int data[], int awal, int akhir)
40 {
41     int q;
42     if (awal < akhir)
43     {
44         q = partisi(data, awal, akhir);
45         quick_sort(data, awal, q);
46         quick_sort(data, q + 1, akhir);
47     }
48 }
```

File `sorting.cpp` merupakan implementasi dari fungsi-fungsi yang telah dideklarasikan pada file `sorting.h`. Berikut adalah penjelasan mengenai alur logika di dalamnya:

1. **Fungsi `tampilkan_larik`:** Fungsi ini melakukan iterasi melalui array dari indeks ke-1 hingga ke- n untuk mencetak seluruh isi data ke terminal. Hal ini memungkinkan pengguna untuk memverifikasi kondisi data pada setiap tahap pengerjaan.
2. **Fungsi `partisi`:** Fungsi ini mengimplementasikan metode partisi (menggunakan skema *Hoare Partition*).
 - a. Fungsi ini memilih elemen pertama sebagai *pivot*.
 - b. Terdapat dua penunjuk (i dan j) yang bergerak saling mendekat untuk mencari elemen yang nilainya lebih besar atau lebih kecil dari *pivot*.
 - c. Jika ditemukan posisi yang tidak sesuai, elemen tersebut akan ditukar (*swap*). Hasil akhirnya adalah pengelompokan data di mana nilai yang lebih kecil dari *pivot* berada di sebelah kiri dan nilai yang lebih besar di sebelah kanan.
3. **Fungsi `quick_sort`:** Fungsi ini menjalankan mekanisme rekursif pada algoritma.
 - a. Setelah fungsi partisi menetapkan titik pisah (*pivot point*), fungsi `quick_sort` akan memanggil dirinya sendiri untuk mengurutkan bagian kiri (data sebelum *pivot*) dan bagian kanan (data sesudah *pivot*) secara terpisah.
 - b. Proses ini akan terus berlanjut hingga sub-array yang diproses hanya tersisa satu elemen (kondisi dasar rekursi), yang menandakan bahwa seluruh data telah terurut dengan benar.

3. Penjelasan *Main Program* (`main.cpp`)



```
1 #include "../include/sorting.h"
2 #include <iostream>
3
4 using namespace std;
5
6 int main()
7 {
8     int i, j, n, data[100];
9
10    cout << "masukkan banyak data= ";
11    cin >> n;
12
13    for (i = 1; i <= n; i++)
14    {
15        cout << "data ke-" << i << " = ";
16        cin >> data[i];
17    }
18
19    cout << "Data sebelum diurut: " << endl;
20    for (j = 1; j <= n; j++)
21    {
22        cout << data[j] << " ";
```

```

23     }
24
25     quick_sort(data, 1, n);
26
27     // hasil pengurutan
28     cout << endl;
29     cout << endl;
30     cout << "hasil pengurutan:\n";
31     tampilkan_larik(data, n);
32
33     return 0;
34 }

```

File `main.cpp` berfungsi sebagai titik awal eksekusi program sekaligus antarmuka interaksi antara pengguna dan sistem. Berikut adalah alur kerja utama dalam file ini:

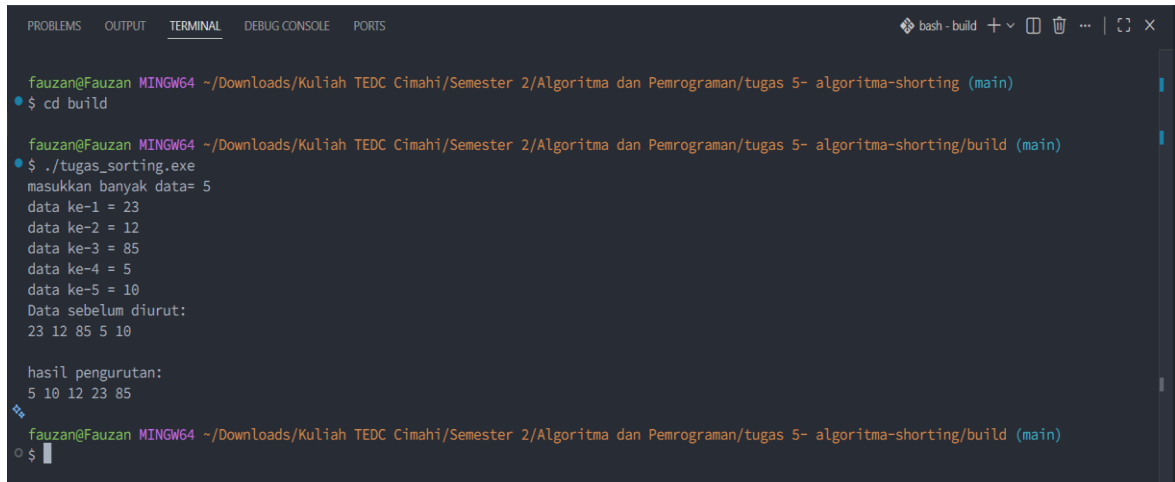
1. **Inisialisasi Data:** Program meminta input dari pengguna mengenai jumlah data (n) yang akan diurutkan.
2. **Pengisian Array:** Program melakukan perulangan (*looping*) untuk menginput nilai ke dalam array. Sesuai dengan spesifikasi tugas, pengisian dimulai dari indeks ke-1 hingga indeks ke- n .
3. **Pemanggilan Fungsi Pengurutan:** Setelah data terkumpul, program memanggil fungsi `quick_sort` yang telah diimpor dari `sorting.h`. Program mengirimkan parameter berupa array, serta batas awal (1) dan batas akhir (n) sebagai acuan proses pengurutan.
4. **Menampilkan Hasil:** Setelah proses pengurutan selesai, program memanggil fungsi `tampilkan_larik` untuk mencetak hasil akhir ke layar, sehingga pengguna dapat melihat bahwa data telah terurut dengan benar dari nilai terkecil ke terbesar.

4. Hasil Pengujian Program

Bagian ini mendokumentasikan hasil eksekusi program setelah melalui proses kompilasi. Pengujian dilakukan dengan memasukkan sekumpulan data acak untuk memastikan algoritma *Quick Sort* bekerja dengan presisi.

Skenario Pengujian:

1. **Jumlah Data (n):** 5
2. **Input Data:** 23, 12, 85, 5, 10
3. **Hasil yang Diharapkan:** Data terurut secara *ascending* (5, 10, 12, 23, 85).



```
fauzan@Fauzan MINGW64 ~/Downloads/Kuliah TEDC Cimahi/Semester 2/Algoritma dan Pemrograman/tugas 5- algoritma-sorting (main)
$ cd build

fauzan@Fauzan MINGW64 ~/Downloads/Kuliah TEDC Cimahi/Semester 2/Algoritma dan Pemrograman/tugas 5- algoritma-sorting/build (main)
$ ./tugas_sorting.exe
masukkan banyak data= 5
data ke-1 = 23
data ke-2 = 12
data ke-3 = 85
data ke-4 = 5
data ke-5 = 10
Data sebelum diurut:
23 12 85 5 10

hasil pengurutan:
5 10 12 23 85

fauzan@Fauzan MINGW64 ~/Downloads/Kuliah TEDC Cimahi/Semester 2/Algoritma dan Pemrograman/tugas 5- algoritma-sorting/build (main)
$
```

Analisis Hasil: Berdasarkan hasil pengujian di atas, program berhasil mengurutkan data masukan dengan benar sesuai dengan logika *Quick Sort*. Indeks array dimulai dari 1 sebagaimana ditetapkan dalam spesifikasi, dan fungsi `tampilkan_larik` berhasil menampilkan urutan data yang telah terorganisir dengan baik dari nilai terkecil ke nilai terbesar.

5. Kesimpulan

Berdasarkan hasil implementasi dan pengujian, algoritma *Quick Sort* terbukti efektif dalam mengurutkan data secara *ascending* dengan pendekatan *divide and conquer*. Penggunaan *header file* (`sorting.h`) berhasil membuat struktur kode menjadi modular, mudah dibaca, dan mudah dikelola sesuai dengan prinsip pemrograman terstruktur yang dipelajari dalam mata kuliah Algoritma dan Pemrograman.